

Types of Hypervisors & Hardware Virtualization



Types of Hypervisors

based on
Hypervisor Implementation Methods

Firmware Hypervisor

Bare Metal Hypervisor

Hosted Hypervisor in Kernel Mode

Hosted Hypervisor in User Mode

Types of Hypervisors

▶ Type 0 hypervisors: firmware

- ❖ Hardware-based solutions providing support for virtual machine creation and management via firmware

▶ Type 1 hypervisors: native or bare metal

- ❖ Operating-system-like software built to provide virtualization

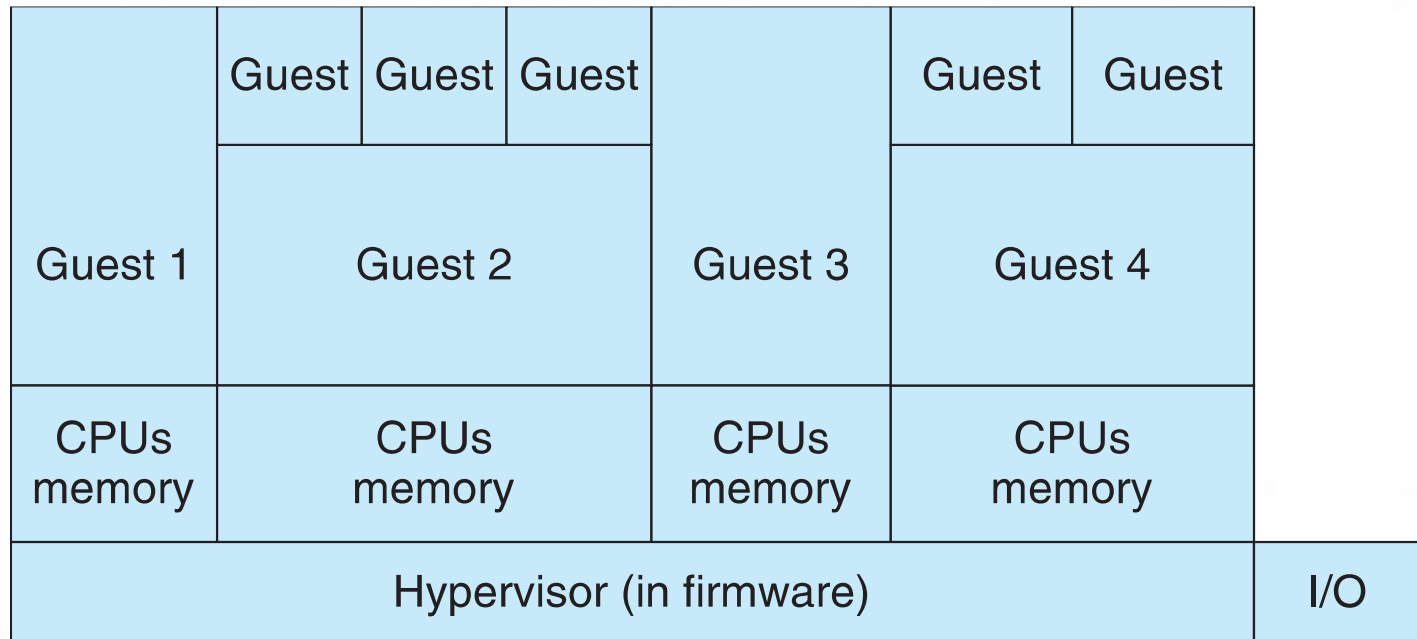
▶ Type 2 hypervisors: hosted in Kernel / User Mode

- ❖ Applications that run on standard operating systems but provide VMM features to guest operating systems

Type 0 Hypervisor : Firmware

- ▶ Old idea, under many names by HW manufacturers
 - ❖ Called “partitions”, “domains”
 - ❖ OS need to nothing special, **VMM is in firmware**
 - ❖ Smaller feature set than other types
 - ❖ Each guest has dedicated HW
- ▶ Sometimes VMM implements a **control partition** running daemons that other guests communicate with for shared I/O
- ▶ Can provide **virtualization-within-virtualization** (guest itself can be a VMM with guests)
 - ❖ Other types have difficulty doing this
- ▶ Ex) **IBM LPARs** and Oracle LDOMs are examples

Type 0 Hypervisor : Firmware



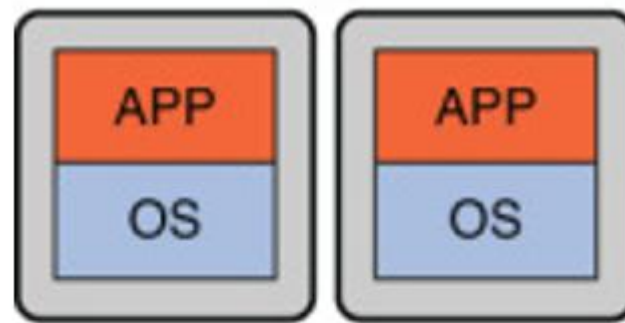
Type 1 Hypervisor : native or bare metal

- ▶ Bare metal hypervisors are software systems that run directly on the host's hardware to control the hardware, and to monitor the guest operating systems.
- ▶ Ex) **Xen**, VMware iESX, Citrix XenServer and Joyent SmartOS

Type 1 Hypervisor : native or bare metal

- ▶ A special purpose operating systems that run natively on HW
 - ❖ Rather than providing system call interface, **create run and manage guest OSes**
 - ❖ Can run on Type 0 hypervisors but not on other Type 1s
 - ❖ Run **in kernel mode**
 - ❖ Guests generally don't know they are running in a VM
 - ❖ **Implement device drivers** for host HW because no other component can
 - ❖ Also provide other traditional OS services like CPU and memory management

Type 1 Hypervisor : native or bare metal



Type 1 Bare Metal

Type 1 Hypervisor : native or bare metal

- ▶ Commonly found in company datacenters

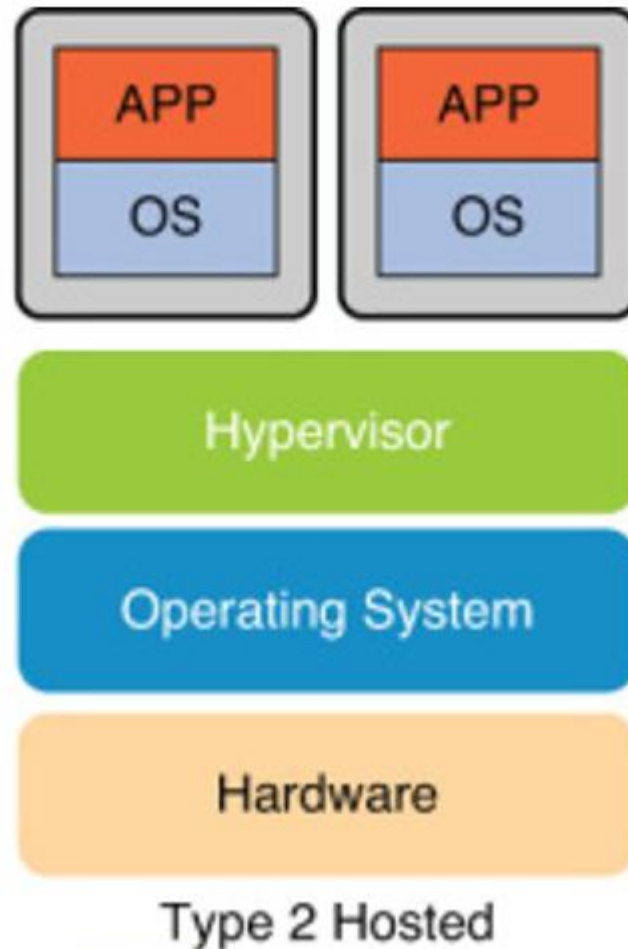
- ❖ In a sense becoming “**datacenter operating systems**”

- Datacenter managers control and manage OSes in new, sophisticated ways by controlling the Type 1 hypervisor
 - **Consolidation of multiple OSes** and apps onto less HW
 - **Move guests** between systems to balance performance
 - **Snapshots and cloning**

Type 2-1 Hypervisor : hosted in Kernel Model

- ▶ Hosted hypervisors are designed to run within a traditional operating system.
- ▶ In other words, a hosted hypervisor **adds a distinct software layer on top of the host operating system**, and the guest operating system becomes a third software level above the hardware.
- ▶ Perform normal **OS duties as well as VMM duties**
- ▶ Ex) Microsoft Windows Server with **HyperV** and RedHat Linux with **KVM**

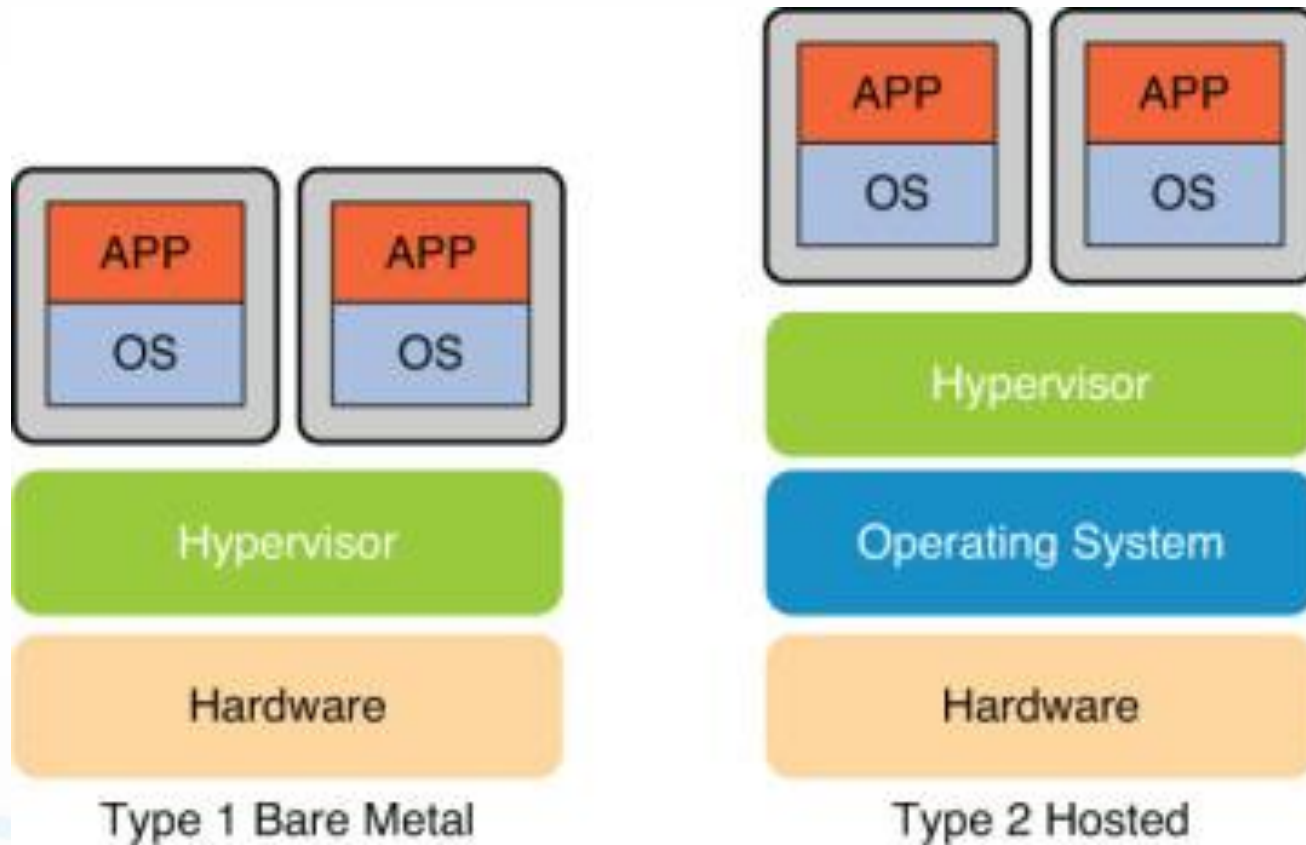
Type 2-1 Hypervisor : hosted in Kernel Model



Type 2 Hypervisor : hosted in User Mode

- ▶ Less interesting from an OS perspective
 - ❖ Very little OS involvement in virtualization
 - ❖ VMM is **simply another process, run and managed by host**
 - Even the host doesn't know they are a VMM running guests
 - ❖ Tend to have **poorer overall performance** because can't take advantage of some HW features
 - ❖ But also a benefit because require **no changes to host OS**
 - Student could have Type 2 hypervisor on native host, run multiple guests, all on standard host OS such as Windows, Linux, MacOS
- ▶ Ex) VMware Workstation and Fusion, Oracle **VirtualBox**, Microsoft Virtual PC, Parallels Desktop, Oracle VM for x86,

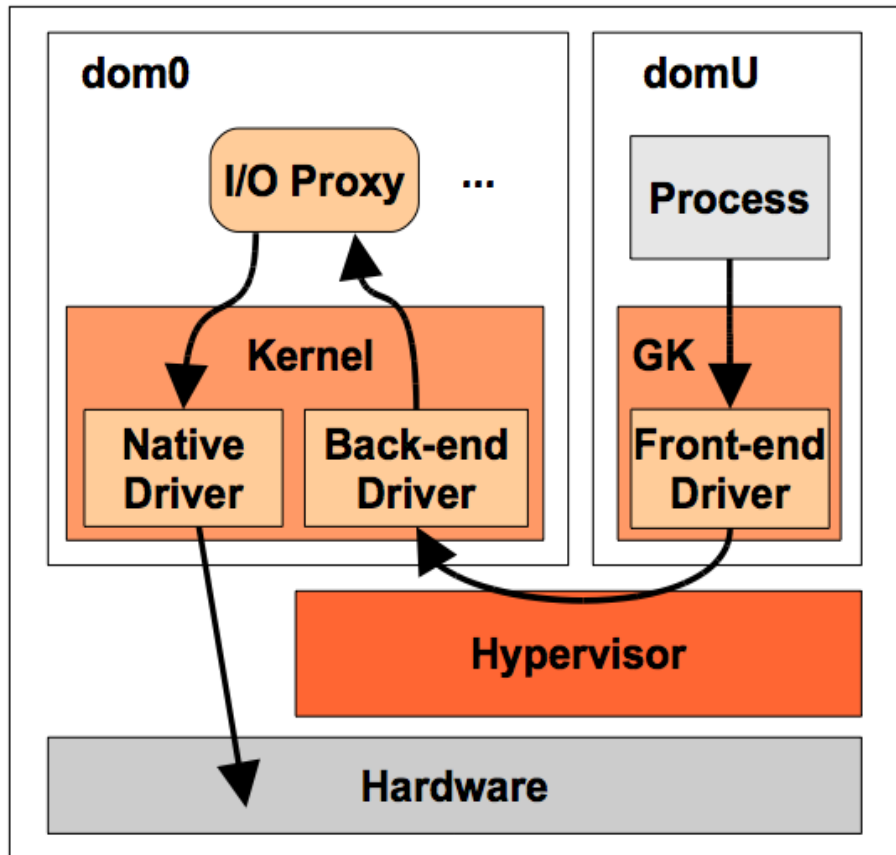
Types of Hypervisors



Types of Hypervisors

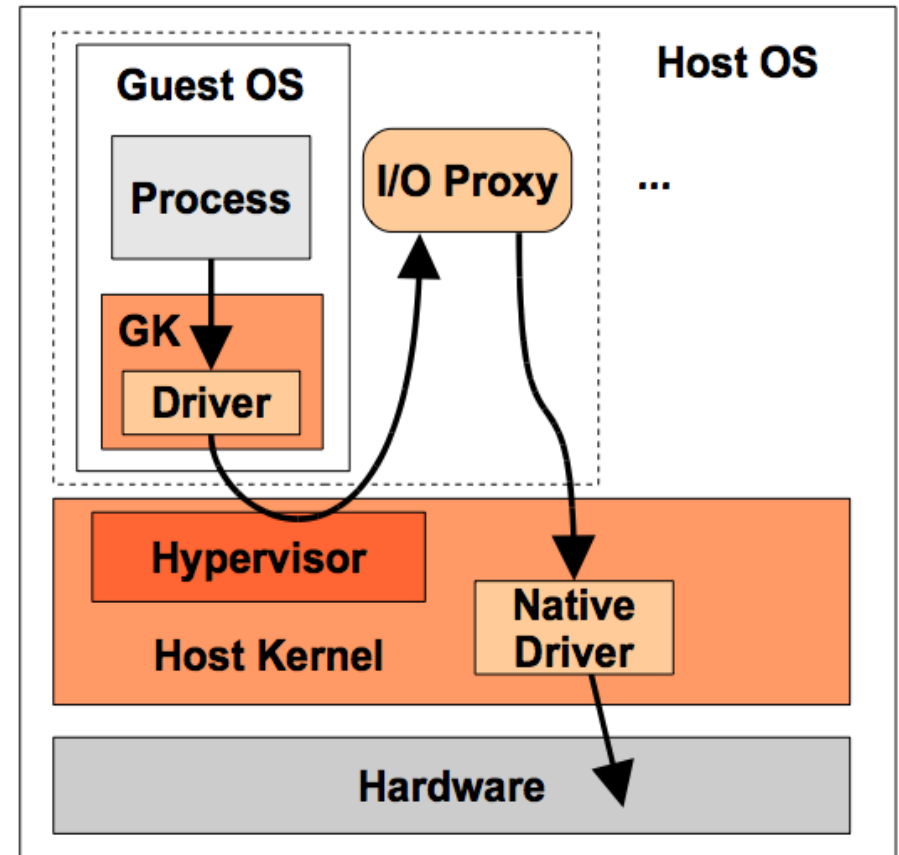
Type1 Hypervisor

Xen



Type2 Hypervisor

KVM



Bochs / Crusoe / QEMU / BIRD / Dynamo

Types of HW Virtualization

based on

How to Handle Privileged Instructions

Full Virtualization

Para Virtualization

Hardware-Assisted Virtualization

시스템 가상화 기법 2가지

에뮬레이션(Emulation)
직접 실행(Direct Execution)



에뮬레이션(Emulation)

- ▶ Q: ARM 아키텍처를 기반으로 만들어진 안드로이드 플랫폼을 어떻게 x86 아키텍처인 PC에서 실행할 수 있을까?
- ▶ A: Emulation
 - 에뮬레이터는 안드로이드 ARM 바이너리 이미지를 스캔
 - 역어셈블 과정으로 ARM 명령어를 추출
 - 추출해 낸 ARM 명령어를 x86에서 동작할 수 있도록 번역
 - 특히, 메모리 테이블을 수정하는 명령어
 - CPU 레지스터 데이터 관리에 대한 명령어
 - 이렇게 번역된 코드를 소프트웨어로 만들어진 가상머신 위에서 동작
- ▶ c: 에뮬레이션은 위와 같이 번역의 과정을 거치므로 당연히 실제 머신이나 디바이스에서 실행하는 것보다 성능이 저하된다.
 - ❖ 번역 가능한 것만 번역할 수 있음.

직접 실행(Direct Execution)

- ▶ Q: 만약 에뮬레이션해야 하는 대상 아키텍처와 실제 구동시킬 머신 혹은 디바이스의 아키텍처가 같을 때도 이런 불필요한 번역 과정이 필요할까?
- ▶ A: 이런 단점을 극복하고자 사용하는 방식이 바로 직접 실행(Direct Execution)방식이다. 바이너리 프로그램을 번역 과정 없이 직접 CPU에서 실행하는, 즉 가상화 환경이 아닌 것처럼 바이너리 프로그램을 CPU가 직접 실행하게 만드는 것
- ▶ C: 불필요하게 번역할 필요가 없으므로 가상화하지 않은 환경과 같은 성능을 보장하는 동시에 가상화를 통한 다양한 이점을 누릴 수 있다. 그러나 직접 실행 방식으로 하이퍼바이저를 구현할 때는 여러 가지 해결해야 할 문제점이 있다.
 - ❖ 대표적인 문제
 - 비특권 모드에서 특권 명령 처리는 어떻게 하는가?

특권 모드 vs. 비특권 모드

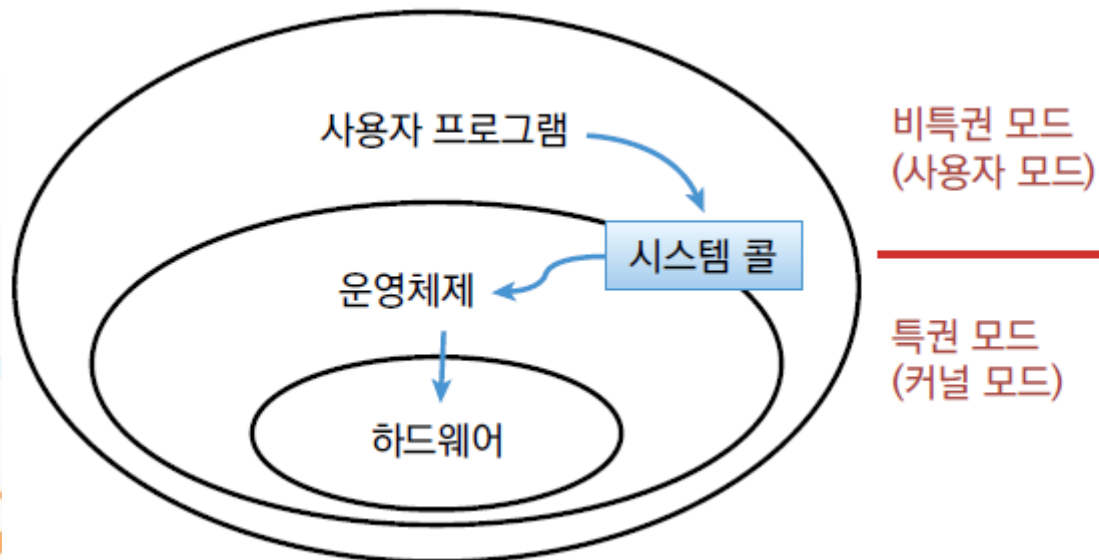
- ▶ 특권 모드 (Privileged mode)

- ❖ e.g. 운영체제 커널

- ▶ 비 특권 모드 (Non privileged mode)

- ❖ e.g. 응용 프로그램

- ▶ 비 특권 모드에서 구동되는 프로그램은 특권 모드의 메모리 영역과 프로세스에 접근할 수 없다.



특권 모드와 비특권 모드

(Privileged vs. Non-Privileged Mode)

- ▶ 운영체제는 시스템 전체를 관장하며, 하드웨어를 보호하고 관리한다.
다시말해 운영체제는 애플리케이션이 하드웨어에 직접 접근할 수 없게 하고, 원하는 서비스를 **시스템 콜**을 통해 요청하도록 규정한다.
 - ❖ 애플리케이션이 파일에서 데이터를 읽으려면 프로그래머는 read()라는 시스템 콜을 호출하며, 운영체제는 디스크에서 직접 데이터를 읽어 애플리케이션에 전달한다.
- ▶ 특권 모드와 비특권 모드를 명확하게 구분하지 않으면 **악의적인 프로그램** 사용 혹은 프로그래밍 실수 때문에 **시스템 전체의 안정성을 위협**할 것이다.
- ▶ 프로그래밍할 때 흔히 볼 수 있는 에러 메시지인 '**Segmentation Fault**'가 바로 여기에 해당하며, **곧바로 애플리케이션이 종료**되므로 전체 시스템에는 큰 영향을 미치지 않는다.

특권 명령과 트랩 (Trap)

▶ 특권 명령

- ❖ 특권 모드에서만 실행 가능한 명령어

▶ 트랩 (trap)

- ❖ 0으로 나누기나 디버거를 위한 중단점 (break point) 등과 같이 **특정 이벤트를 만나면 프로그램의 제어권이 트랩 핸들러 (트랩 사건을 처리하는 루틴)로 넘어가는 것**을 의미

▶ 비특권 모드에서 특권 명령을 실행할 때 시스템에 트랩 발생

▶ e.g. 응용 프로그램의 하드웨어 접근

- ❖ x86 아키텍처의 (입·출력) 관련 명령어는 특권 명령

Trap & Emulation

(전통적 Hypervisor 구현방법)



- ▶ 하이퍼바이저는 특권 모드에서 구동
- ▶ 운영체제는 비특권 모드에서 구동
- ▶ Trap & Emulation 방식의 하이퍼바이저 구동
 - ❖ 비특권 모드에서 동작하는 운영체제가 특권 명령을 실행할 때마다 트랩 발생
 - ❖ 하이퍼바이저로 제어권이 넘어감
 - ❖ 하이퍼바이저는 적절히 관련 루틴을 에뮬레이트(실행)하고 다시 제어권을 운영체제로 넘겨줌

Trap & Emulation 방식의 문제점



- ▶ 트랩 그 자체로는 시스템 오버헤드가 매우 큼
- ▶ x86 아키텍처는 트랩과 에뮬레이트 방식만으로 하이퍼바이저 구현 불가능
 - ❖ 특권 명령과 비특권 명령의 경계가 모호한 명령어가 있었기 때문임
 - ❖ 약 17개 명령어가 트랩과 에뮬레이트 방식으로 하이퍼바이저를 구현하기 어렵게 만든다고 알려짐

CPU Direct Execution - 바이너리 변환과 하이퍼 콜-

▶ Trap & Emulation 방식의 문제점 해결책으로 등장

- ❖ VMWare : Binary Translation

- ❖ Xen : Hypercall

▶ Binary Translation => Full Virtualization

- ❖ 운영체제의 코드 이미지를 하이퍼바이저가 스캔해 문제의 소지가 있는 17 Sensitive 명령어를 찾아내고 검출

- ❖ 해당 명령어를 적절히 수정하여 잘 동작할 수 있는 코드를 만들어 실행

- CPU에서 직접 실행하는(Directe Execution) 방식이지만 중간에 하이퍼바이저가 번역(Binary Translation)하는 과정이 추가

CPU 직접 실행 방식

- 바이너리 변환과 하이퍼 콜-

▶ Hypercall => Para Virtualization

❖ 반가상화 기술의 일종

❖ 직접 운영체제의 소스를 수정하여 문제 소지가 있는 명령어 제거

➤ 하이퍼바이저로 호출할 수 있는 하이퍼 콜로 바꿈

➤ 문제 소지가 있는 명령어가 실행될 시점에 하이퍼바이저로 제어권 넘어감

➤ 하이퍼바이저는 게스트 운영체제의 요청을 받아서 적절히 처리

➤ 다시 게스트 운영체제로 제어권을 넘겨줌

❖ 현재는 많은 부분이 Linux Kernel에 내장됨.

➤ 예전만큼의 수정 불필요

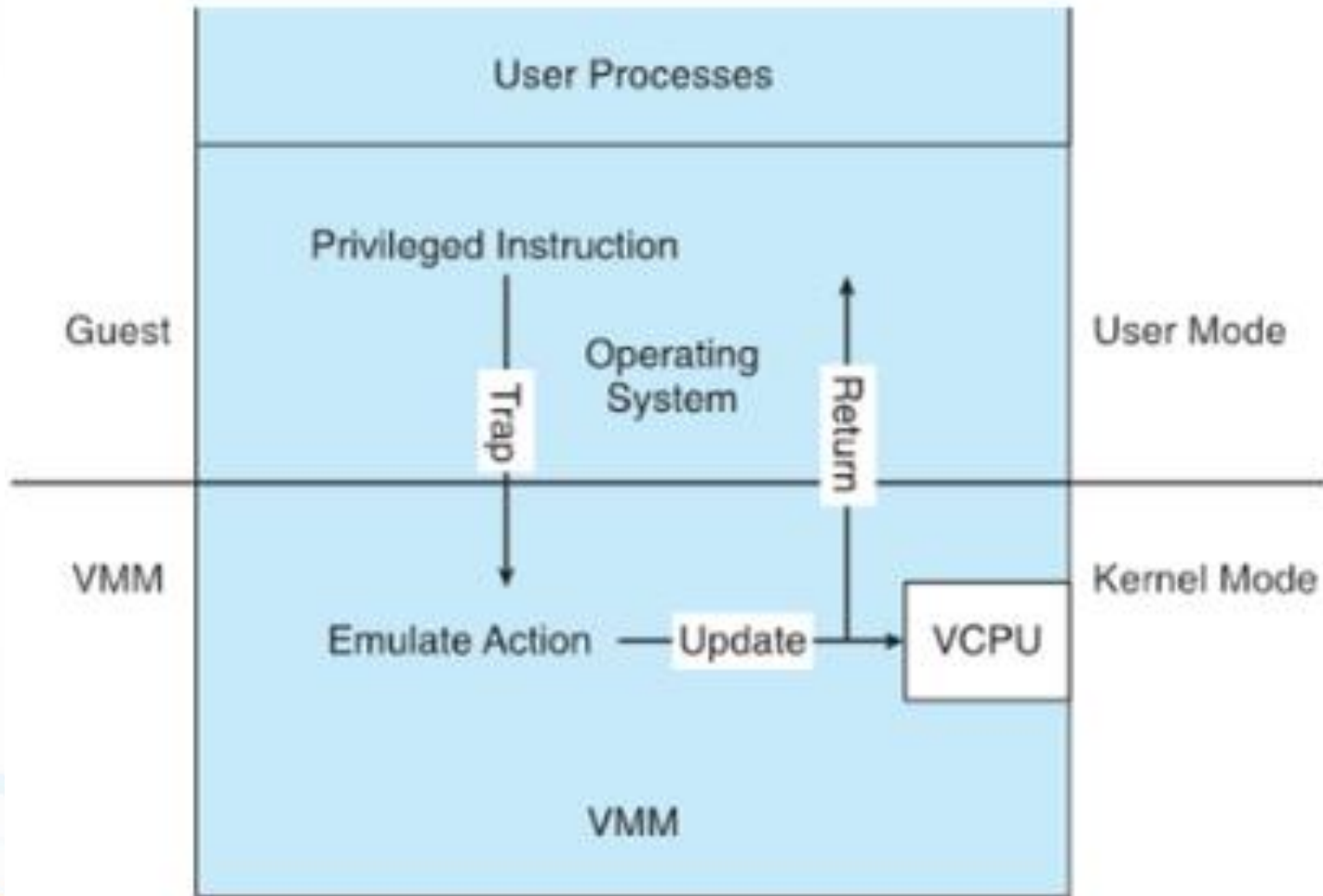
Full Virtualization

- ▶ “Hardware is functionally identical to underlying architecture.”
- ▶ Virtualization Methods
 - ❖ **Trap and Emulate** for Privileged instructions and
 - ❖ **Binary Translation** for Sensitive instructions (Not-Privileged))
- ▶ Advantage:
 - ❖ Guest OS will run without any changes to source code.
- ▶ Disadvantage:
 - ❖ Complex, usually slower than para-virtualization.

Trap & Emulate for Privileged Instructions

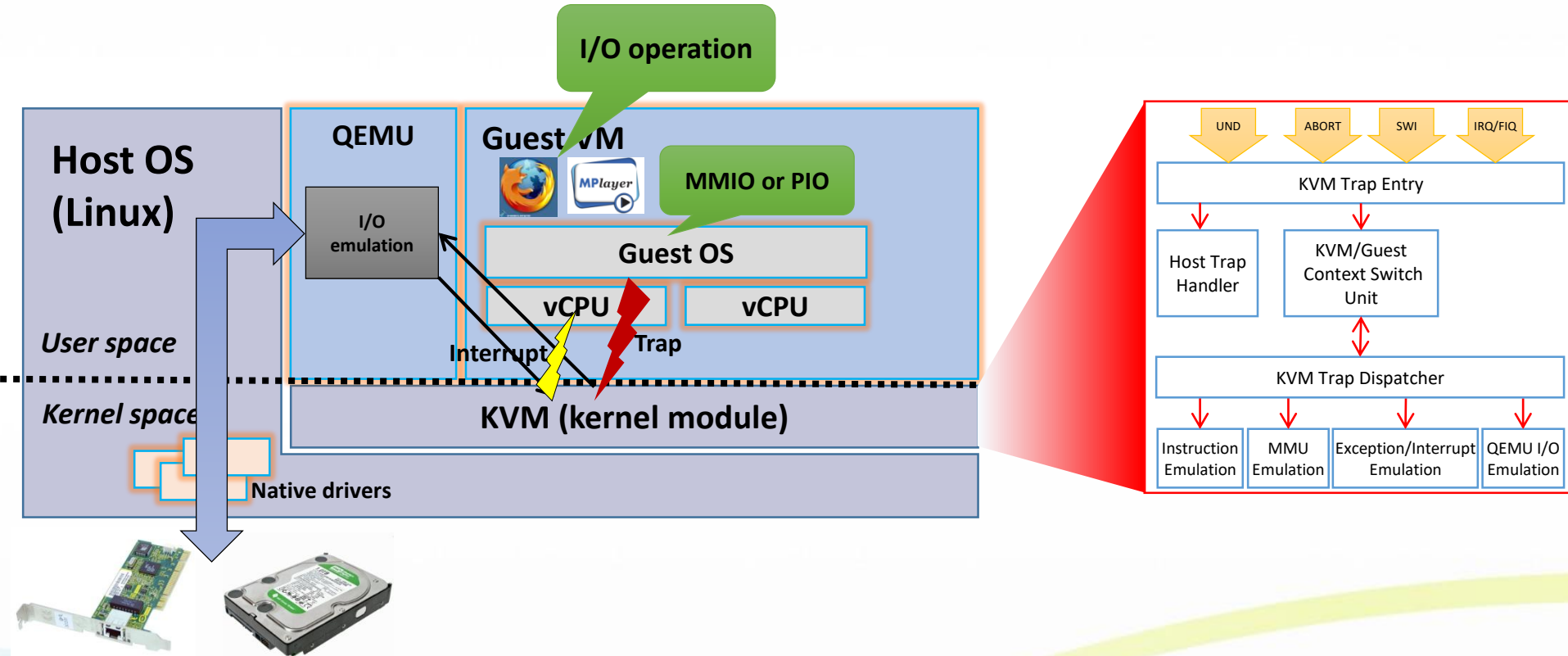
- ▶ Modern VMMs based around trap-and-emulate.
- ▶ **When a guest OS executes a privileged instruction, control is passed to VMM** (VMM “traps” on instruction), which decides how to handle instruction.
- ▶ VMM generates instructions **to handle trapped instruction (emulation) by itself or through QEMU.**
- ▶ Non-privileged instructions do not trap (system stays in guest context).

Trap & Emulate



Trap and Emulated Full Virtualization

► KVM + QEMU I/O Emulation



Source: <https://www.slideshare.net/HwanjuKim/5io-virtualization>

Source: <https://www.slideshare.net/PeterChang6/armvisor-linux-symposium-2012-14155367>

Trap-and-Emulate Problems

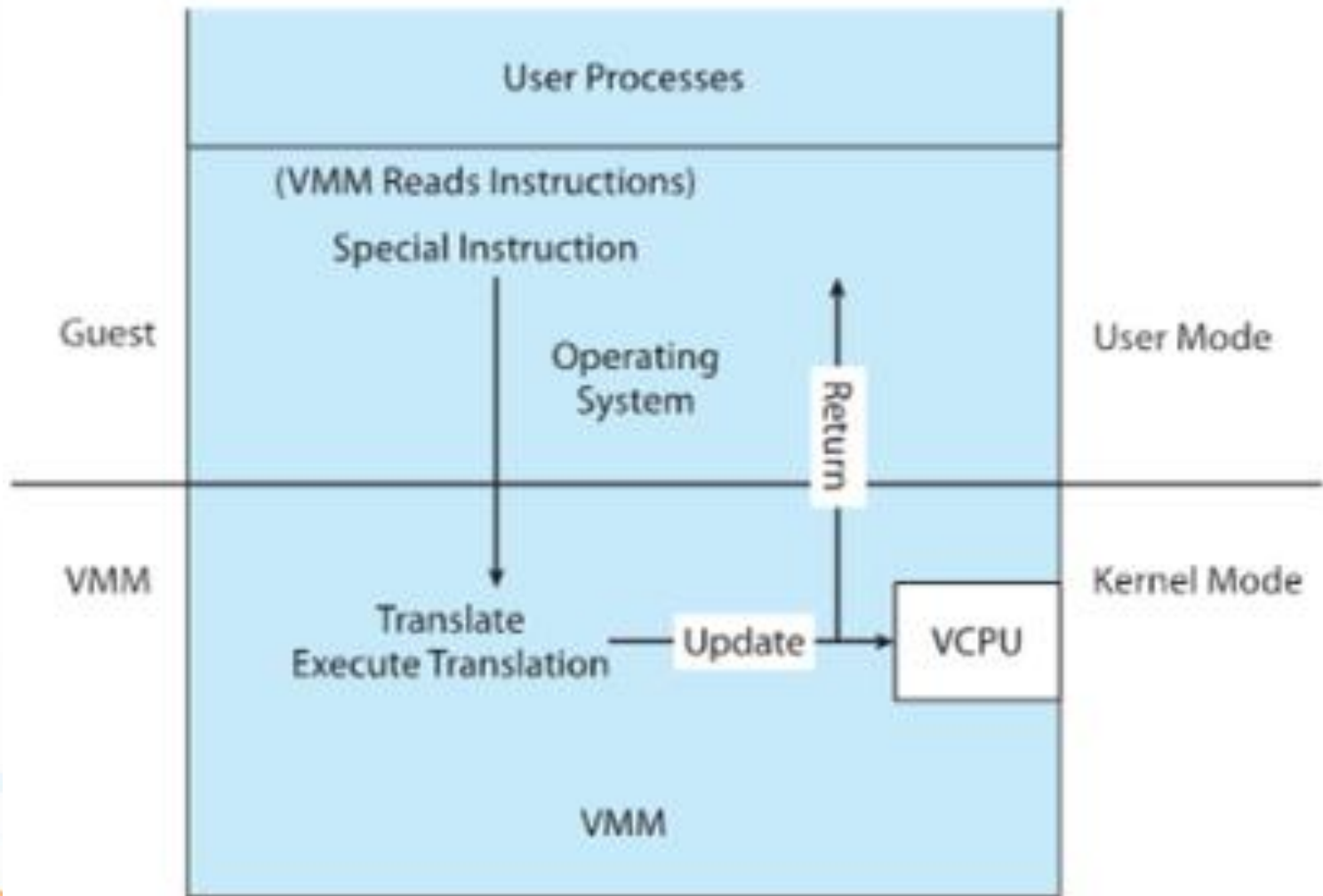
- ▶ Trap-and-emulate is expensive
 - ❖ Requires context-switch from guest OS mode to VMM.
- ▶ x86 is not trap-friendly
 - ❖ Guest's CPL Privilege Level is visible in hardware registers; cannot change it in a way that the guest OS cannot detect.
 - ❖ Some instructions, called 17 **Sensitive Instructions**, are not privileged, but access privileged systems (page tables, for example) .

Binary translation

- ▶ **Sensitive Instructions** can be implemented through dynamic binary translation **by VMM**.
 - ❖ Translated code is grouped and stored in translation caches (TCs).
 - ❖ **Callout method replaces traps with stored emulation functions.**
 - ❖ In-TC emulation blocks are even more efficient.
 - ❖ Adaptive binary translation rewrites translated blocks to minimize PTE traps .
- ▶ Direct execution of user-space code further reduces overhead.

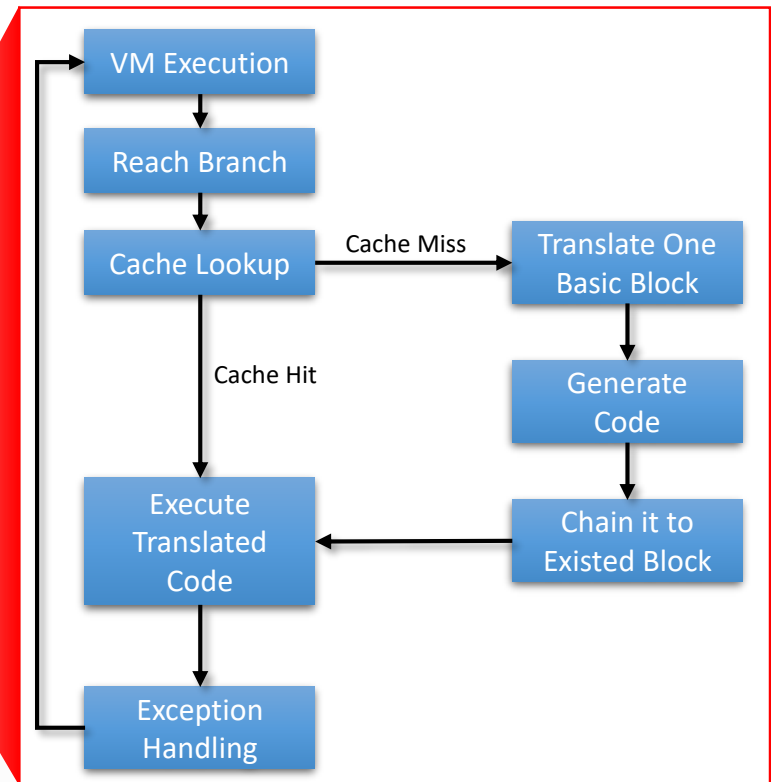
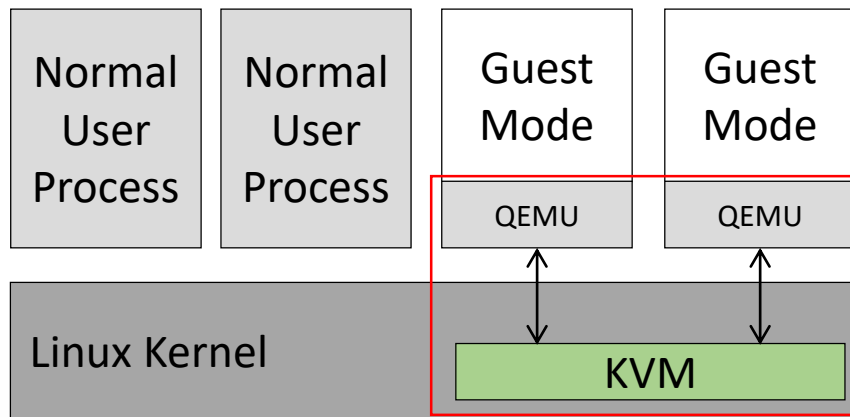
* PTE: Page Table Entry

Binary translation (2)



Binary Translated Full Virtualization

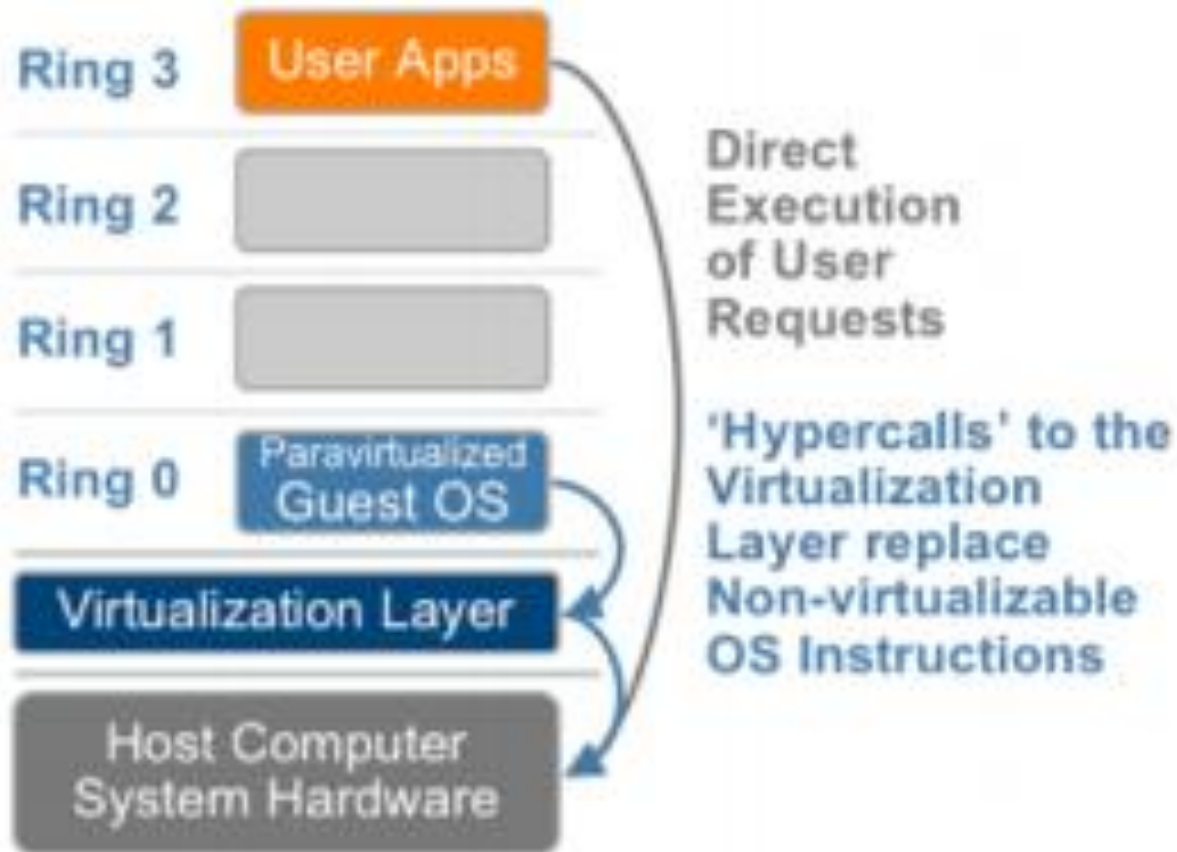
► KVM + QEMU [Guest(Linux/arm) → Host(Linux/x86)]



Para-Virtualization (1)

- ▶ **Guest had to be modified** to use run on paravirtualized VMM
 - ❖ Replace certain unvirtualized sections of OS code with virtualization-friendly code.
- ▶ Xen, leader in paravirtualized space, adds several techniques
 - ▶ Guest uses **hypercall** (call to hypervisor) when page-table changes needed
 - ▶ But on modern CPUs Xen no longer requires guest modification
 - > no longer paravirtualization (hardware-assisted Virtualization)
- ▶ Advantages: easier, lower virtualization overhead
- ▶ Disadvantages: requires modifications to guest OS

Para-Virtualization (2)



Hypercall

- ▶ Xen occupies privilege level 0; guest OS occupies privilege level 1.
- ▶ OS code is modified so that high-privilege calls (hypercalls) are made to and trapped by Xen.
- ▶ **Xen traps guest OS instructions using table of exception handlers.**
 - ❖ Frequently used handlers (e.g., system calls) have special handlers that allow guest OS to bypass privilege level 0.
 - ❖ Approach does not work with page faults.
 - ❖ Handlers are vetted by Xen before being stored.

하드웨어 지원

▶ HVM (Hardware assisted Virtual Machine)

- ❖ Xen, KVM 등에서 가능한 방식

▶ 가상화 기능을 CPU에 추가

- ❖ AMD : SVM (Secure Virtual Machine), 인텔 : VT-x

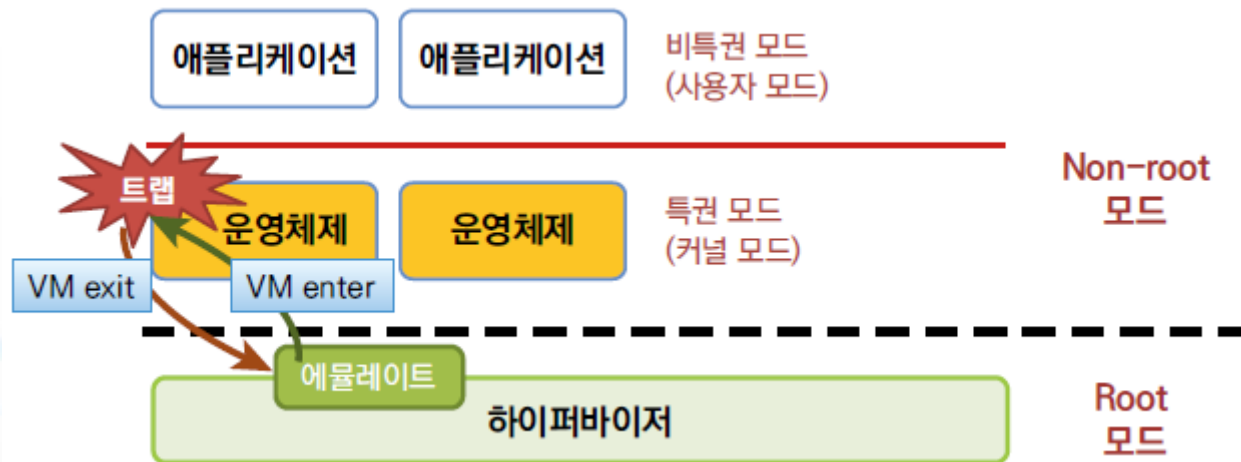
- ❖ ARM 프로세서에도 가상화 기능 추가

▶ 핵심

- ❖ 게스트 운영체제의 소스 코드를 수정(Hypercall)하거나 바이너리 변환 방법(Binary Translation) 없이도 트랩과 에뮬레이션(Trap Emulation) 방식으로 하이퍼바이저를 구현할 수 있도록 하드웨어적인 기능 추가 (CPU에 기능 추가)

Intel VT-x의 예

- ▶ 추가로 하이퍼바이저를 위한 root 모드와 게스트 운영체제를 위한 non-root 모드 등장
 - ❖ 하이퍼바이저는 root 모드에서 동작
 - ❖ 게스트 운영체제는 non-root 모드에서 동작
 - 게스트 운영체제는 non-root 모드의 특권 모드에서 동작
 - 애플리케이션은 non-root 모드의 비특권모드에서 동작



Intel VT-x의 예

- ▶ 문제의 소지가 있는 명령어(Privileged/Sensitive instructions)가 non-root 모드에서 실행되면 트랩 발생
- ▶ root 모드로 변경되어 하이퍼바이저로 제어권이 넘어가도록 하드웨어적으로 구현
- ▶ 문제 있는 명령어는 자동으로 트랩이 발생
- ▶ 바이너리 변환이나 하이퍼 콜 등의 도움 없이 해당 명령어를 적절히 처리할 수 있음



Hardware-Assisted Virtualization

- ▶ **Privileged and sensitive calls are set to automatically trap to the hypervisor**, removing the need for either binary translation or paravirtualization.
- ▶ The guest state is stored in Virtual Machine Control Structures (VT-x) or Virtual Machine Control Blocks (AMD-V).

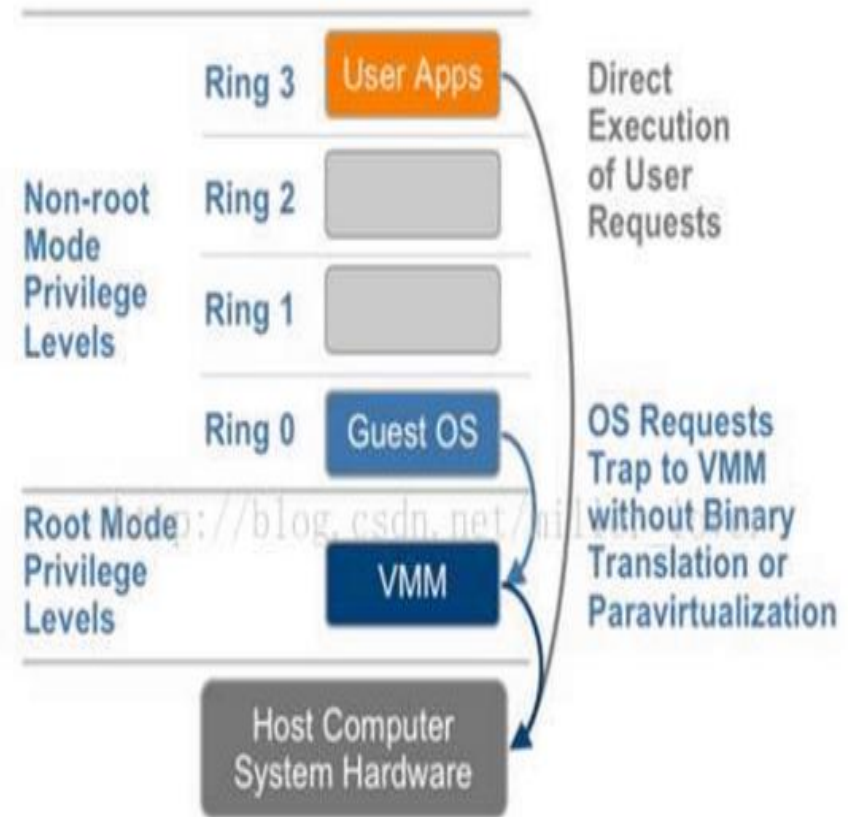


Figure 7 – The hardware assist approach to x86 virtualization

Hardware-Assisted Virtualization

▶ Advantages

- ❖ No binary translation
- ❖ No OS modification
- ❖ Simplifying VMM
 - KVM was born and included in Linux mainline in 2007
 - Vmware, Xen, etc. adopt HW-assisted virtualization
 - Several lightweight VMMs were implemented
 - lguest, tiny VMM, ...
- ❖ Contributions to wide adoption of virtualization

▶ Disadvantages

- ❖ More expensive trap (VMEXIT)
- ❖ Outdating sophisticated and clever SW techniques ☹

GPU PCI Passthrough within the Xen Hypervisor

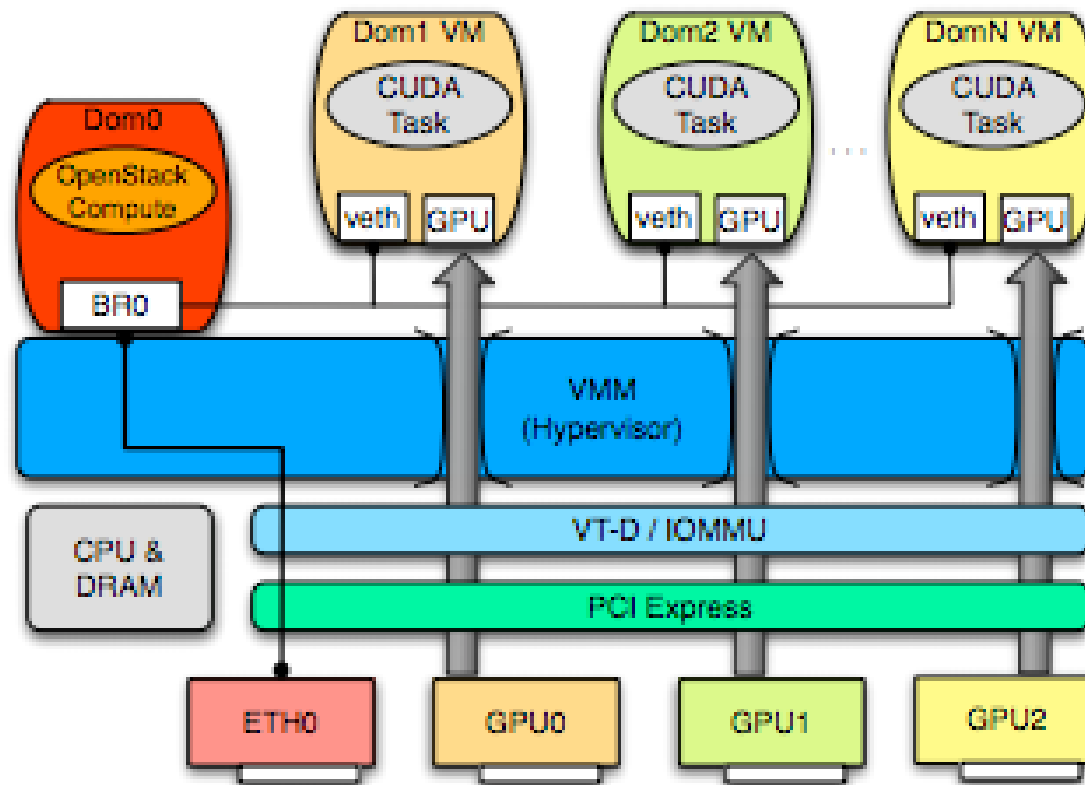


Figure 1: GPU PCI Passthrough within the Xen Hypervisor

Thank You!!



Any Questions ??